



# Stripe Checkout & 3D Secure (3DS)

সম্পূর্ণ ভিজুয়াল গাইড — একজন নতুন Developer এর জন্য বাংলায়

Project: LifeChoice Backend | Backend: Django REST Framework | Payment Gateway: Stripe

## সূচিপত্র (Table of Contents)

1. Stripe Checkout ফ্লো — ধাপে ধাপে (Frontend → Backend → Stripe → Webhook → Success)
2. 3D Secure (3DS) ফ্লো — কীভাবে কাজ করে
3. Backend Implementation Guide (Django/Python)
4. Frontend Implementation Guide (React/JavaScript)
5. সম্পূর্ণ End-to-End Flow Diagram
6. Test Cards এবং Scenarios
7. Common Errors এবং Solutions
8. Quick Reference Card

## ১. Stripe Checkout ফ্লো — সম্পূর্ণ ধাপে ধাপে

Stripe Checkout হলো Stripe এর **hosted payment page** — মানে card details নেওয়ার UI Stripe নিজেই বানিয়ে দেয়। তোমার Frontend কে শুধু user কে সেই page এ পাঠাতে হয়, আর পেমেন্ট শেষে Stripe নিজেই একটা webhook পাঠিয়ে জানিয়ে দেয়। নিচে প্রতিটি ধাপ actor (কে কী করছে), API call, এবং data সহ দেখানো হলো।

## Actor Legend



1



### Frontend থেকে Backend API call

User "Buy Now" বাটনে ক্লিক করলে Frontend Backend কে API call করে — শুধু কোন item কিনতে চায় সেই ID পাঠায়, কোনো card details পাঠায় না।

```
POST /api/v2/enrollment/
Authorization: Bearer <jwt-token>
Content-Type: application/json

{
  "micro_credential_id": 5
}
```

2



### Backend এ `stripe.checkout.Session.create()` কল

Backend প্রথমে DB থেকে price ও Stripe secret key নেয়, তারপর Stripe এর API কে একটা Checkout Session তৈরি করতে বলে। এখানে product name, price, success/cancel URL এবং metadata (user\_id, item\_id) পাঠানো হয়।

```
stripe.checkout.Session.create(
  payment_method_types=['card'],
  line_items=[{
    'price_data': {
      'currency': 'usd',
      'unit_amount': 1499, # cents ($14.99)
      'product_data': {'name': 'Micro-Credential X'}
    },
    'quantity': 1,
  }],
  mode='payment',
  success_url='https://app.com/success?session_id={CHECKOUT_SESSION_ID}',
  cancel_url='https://app.com/cancel',
  metadata={'user_id': '12', 'micro_credential_id': '5'}
)
```

3



### Stripe থেকে session.url রিটার্ন

Stripe সার্ভার একটা unique session তৈরি করে তার URL এবং session ID ফেরত দেয়। এই URL টাই হলো hosted checkout page — যেখানে user card দেবে।

```

{
  "id": "cs_test_a1B2c3D4...",
  "url": "https://checkout.stripe.com/pay/cs_test_a1B2c3D4...",
  "payment_status": "unpaid",
  "status": "open"
}
  
```

4



### User কে Stripe Hosted Page এ redirect

Backend সেই checkout\_url টা Frontend কে ফেরত দেয়। Frontend সাথে সাথে ব্রাউজারকে সেই URL এ পাঠিয়ে দেয় (redirect) — এখন user Stripe এর নিজস্ব ডোমেইনে (checkout.stripe.com) চলে যায়।

```

// Backend Response
{ "success": true, "checkout": { "checkout_url": "https://checkout.stripe.com/pay/cs_test_...", "session_id": "cs_test_..." } }

// Frontend
window.location.href = data.checkout.checkout_url;
  
```

5



### User কার্ড দিয়ে পেমেন্ট করে

Stripe এর hosted page এ user card number, expiry, CVC দেয়। Stripe নিজেই card validate করে এবং প্রয়োজনে bank এর সাথে 3DS authentication করে (এই অংশ Section ২ তে বিস্তারিত)।

6



### Stripe checkout.session.completed webhook পাঠায়

পেমেন্ট সফল হলে Stripe সরাসরি তোমার সার্ভারের webhook endpoint এ একটা HTTP POST পাঠায় (browser থেকে না — Stripe এর সার্ভার থেকে সরাসরি)। এতে signature header থাকে security এর জন্য।

```
POST https://api.yourapp.com/enrollment/stripe/webhook/
Stripe-Signature: t=175xxx,v1=abcd1234...

{
  "type": "checkout.session.completed",
  "data": { "object": {
    "id": "cs_test_a1B2c3D4...",
    "payment_status": "paid",
    "amount_total": 1499,
    "metadata": {"user_id": "12", "micro_credential_id": "5"}
  }}
}
```

7



### Backend webhook receive করে enrollment/access তৈরি করে

Backend signature verify করে, metadata থেকে user ও item বের করে, PaymentTransaction save করে এবং MCAccess/DegreeAccess তৈরি করে user কে access দেয়। এরপর confirmation email পাঠায়।

```
1. signature verify ✓
2. PaymentTransaction.objects.create(status='completed', ...)
3. MCAccess.objects.create(user=..., micro_credential=..., is_active=True)
4. EmailService.send_user_enrollment_confirmation(...)
5. return HttpResponse(status=200) # Stripe কে জানানো "পেয়েছি"
```

8



### User কে success page এ redirect

Webhook এর সাথে সমান্তরালে, ব্রাউজারে Stripe user কে তোমার নির্ধারিত `success_url` এ পাঠিয়ে দেয়, সাথে `session_id` attach করে দেয়। Frontend সেই session\_id দিয়ে backend থেকে যাচাই করে নিতে পারে পেমেন্ট আসলেই সফল হয়েছে কিনা।

```
Redirect → https://app.com/enrollment/success?
session_id=cs_test_a1B2c3D4...&type=mc
```

### ⚠️ গুরুত্বপূর্ণ পয়েন্ট

Webhook (ধাপ ৬-৭) এবং Success Redirect (ধাপ ৮) — দুটো

#### আলাদা এবং সমান্তরাল

ঘটনা। কখনো কখনো webhook একটু দেরিতে আসতে পারে। তাই success page এ শুধু "ধন্যবাদ" মেসেজ দেখানো উচিত, আর আসল access grant হয় webhook থেকেই — কখনোই শুধু redirect এর উপর ভরসা করে access দেওয়া ঠিক না।

## ২. 3D Secure (3DS) ফ্লো — কীভাবে কাজ করে

### 3DS কী এবং কেন দরকার

**3D Secure (3DS)** মানে "3-Domain Secure" — এটা card পেমেণ্টে একটা অতিরিক্ত authentication layer, যেখানে card এর আসল মালিক OTP বা bank app এর মাধ্যমে নিজেকে যাচাই করে। এটা মূলত ৩টা domain (Merchant, Bank/Issuer, Card Network — Visa/Mastercard) এর মধ্যে communication এর মাধ্যমে কাজ করে।

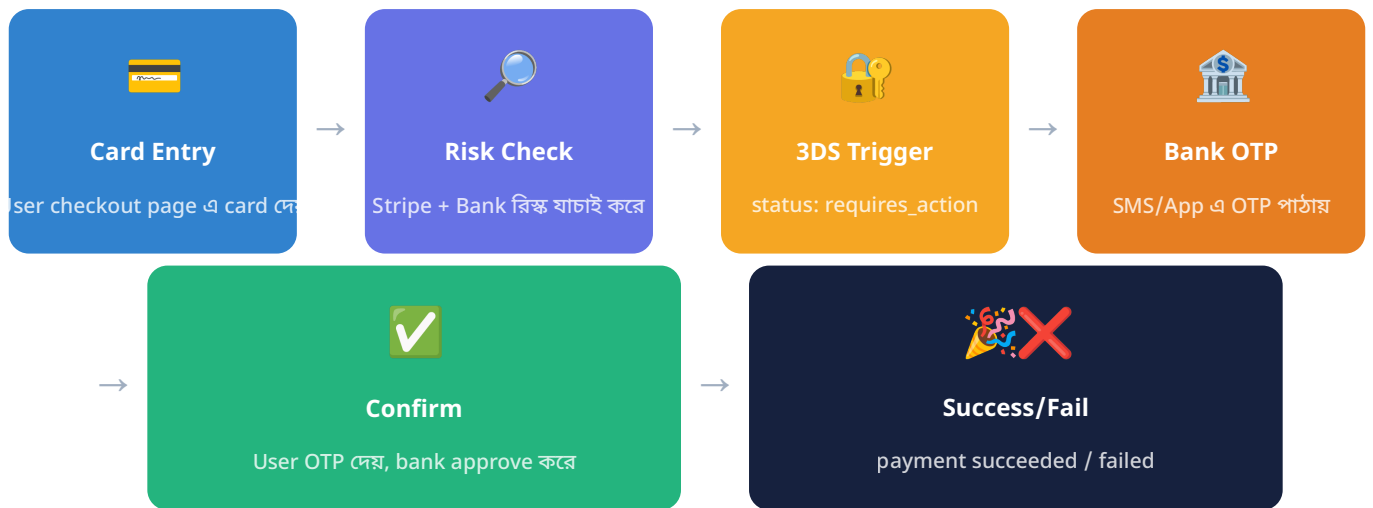
| টার্ম                | ব্যাখ্যা                                                                                                                |
|----------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>PSD2</b>          | Payment Services Directive 2 — ইউরোপীয় ইউনিয়নের একটি regulation যা online পেমেণ্টে অতিরিক্ত security বাধ্যতামূলক করে। |
| <b>SCA</b>           | Strong Customer Authentication — PSD2 এর অংশ, যেখানে কমপক্ষে ২টি factor (যেমন card + OTP) দিয়ে যাচাই করতে হয়।         |
| <b>EU Compliance</b> | ইউরোপের ভেতরের যেকোনো merchant/customer পেমেণ্টে SCA মানতে বাধ্য — নাহলে bank পেমেণ্ট reject করতে পারে।                 |

### কেন দরকার?

3DS ছাড়া card চুরি হলে বা অনুমতি ছাড়া ব্যবহার হলে ঠেকানো কঠিন। OTP/Authentication ধাপ যুক্ত করলে fraud অনেক কমে যায় — কারণ শুধু card number জানলেই আর টাকা কাটা যায় না, real owner এর phone/app এ approval লাগে।

### Stripe Checkout এ 3DS Automatically কীভাবে Handle হয়

Stripe Checkout ব্যবহার করলে তোমাকে 3DS নিয়ে **কোনো কোড লিখতে হয় না**। Stripe নিজে থেকেই bank এর সাথে communication করে বুঝে নেয় card টার 3DS দরকার কিনা, এবং দরকার হলে hosted page এর ভেতরেই OTP challenge দেখিয়ে দেয়।



### ✖ `requires_action` Status মানে কী

#### `requires_action`

এটা Stripe `PaymentIntent` এর একটা status, যার মানে — "পেমেন্ট এখনো complete হয়নি, কারণ user কে extra authentication (3DS) দিতে হবে।" এই status এলে Stripe Checkout হোস্টেড পেজ নিজেই bank এর challenge/OTP পেজ iframe আকারে দেখিয়ে দেয়। Checkout Session ব্যবহার করলে এটা তোমার Frontend/Backend কোডে দেখা লাগে না — কিন্তু যদি তুমি raw `PaymentIntent` API ব্যবহার করো (V1 approach), তখন তোমার Frontend কে নিজে `stripe.confirmCardPayment()` বা `stripe.handleCardAction()` কল করতে হয়।

### 📱 OTP/Authentication Page কোথা থেকে আসে

OTP page টা আসলে **তোমার কোম্পানি বা Stripe বানায় না** — এটা user এর নিজের bank/card issuer এর সিস্টেম থেকে আসে (যেমন ACS - Access Control Server)। Stripe শুধু সেই page টাকে একটা secure iframe/redirect এর মাধ্যমে Checkout page এর মধ্যে দেখিয়ে দেয়।

**1 Stripe → Card Network (Visa/Mastercard)**

Stripe card network কে জিজ্ঞেস করে এই card এর bank কে challenge পাঠাতে হবে কিনা।

**2 Card Network → Issuing Bank (ACS)**

Card network request টা bank এর Access Control Server (ACS) এ ফরওয়ার্ড করে।

**3 Bank ACS → User**

Bank এর সিস্টেম SMS OTP পাঠায়, অথবা bank এর mobile app এ push notification পাঠায় approve করার জন্য।

**4 User → Bank ACS → Stripe**

User OTP লিখলে বা app এ approve করলে, bank সেই ফলাফল Stripe কে জানায়।

🚩 **3DS সফল হলে কী হয়, ব্যর্থ হলে কী হয়**

✓ **3DS সফল হলে**

- PaymentIntent status `requires_action` থেকে `succeeded` এ পরিবর্তন হয়
- Stripe `checkout.session.completed` webhook পাঠায়
- Backend enrollment/access তৈরি করে
- User success page এ redirect হয়

✗ **3DS ব্যর্থ হলে**

- PaymentIntent status `requires_payment_method` এ চলে যায় (আরেকটা card দিতে হবে)
- Stripe `payment_intent.payment_failed` event পাঠাতে পারে
- Checkout page এ error দেখায়: "Your card was declined"
- User চাইলে আবার চেষ্টা করতে পারে (নতুন card বা retry OTP)

🔄 **Full 3DS Flow Diagram — টেক্সট আকারে**

Card → 3DS Trigger → Bank OTP Page → User confirms →  Success →  
webhook → access grant

└─  Fail →  
error shown → retry



## ৩. Backend Implementation Guide (Django/Python)

### 3.1 Checkout Session তৈরি করা

নিচে `stripe.checkout.Session.create()` এর সব গুরুত্বপূর্ণ parameter সহ একটা সম্পূর্ণ ভিউ দেখানো হলো:

```
import stripe
from rest_framework import status
from django.conf import settings

def create_checkout_session(request):
    # ১. DB থেকে Stripe secret key নাও
    stripe_cfg = get_active_stripe_config()
    stripe.api_key = stripe_cfg.get('STRIPE_SECRET_KEY')

    # ২. Price বের করে DB থেকে
    current_price = MicroCredentialPricing.get_current_price() # যেমন 14.99
    amount_cents = int(float(current_price) * 100) # 1499

    # ৩. success_url ও cancel_url সেট করে
    success_url = settings.PAYMENT_SUCCESS_URL # 'https://app.com/enrollment/success'
    cancel_url = settings.PAYMENT_CANCEL_URL # 'https://app.com/enrollment/cancel'

    # ৪. Checkout Session তৈরি করে
    session = stripe.checkout.Session.create(
        payment_method_types=['card'], # কোন কোন payment method
        allow
        line_items=[{
            'price_data': {
                'currency': 'usd',
                'unit_amount': amount_cents, # cents এ, টাকায় না
                'product_data': {
                    'name': micro_credential.credentials,
                    'description': f'Lifetime access:
{micro_credential.credentials}',
                },
            },
            'quantity': 1,
        }],
        mode='payment', # one-time পেমেন্ট
        (subscription হলে 'subscription')
        success_url=f'{success_url}?session_id={{CHECKOUT_SESSION_ID}}&type=mc',
        cancel_url=f'{cancel_url}?type=mc',
        customer_email=request.user.email, # optional - pre-fill
```

```

email
    metadata={
# ⚠️ webhook এ এই data ফেরত আসবে
        'user_id': str(request.user.id),
        'user_email': request.user.email,
        'micro_credential_id': str(micro_credential_id),
        'transaction_type': 'pay_per_credential',
    },
    expires_at=int(time.time()) + 1800,          # optional – 30 মিনিট পর
    expire
)

return Response({
    'checkout_url': session.url,
    'session_id': session.id,
})

```

### 💡 metadata কেন গুরুত্বপূর্ণ?

`metadata` হলো একমাত্র জায়গা যেখান দিয়ে তুমি নিজের প্রয়োজনীয় তথ্য (`user_id`, `item_id`, `transaction_type`) Stripe এর সাথে সংযুক্ত রাখতে পারো। Webhook যখন `checkout.session.completed` event পাঠায়, তখন এই একই metadata ফেরত আসে — এভাবেই backend বুঝতে পারে কোন user কী কিনেছে।

**SUCCESS\_URL এবং CANCEL\_URL কীভাবে সেট করতে হয়**

| Field                    | উদাহরণ                                                                                 | নোট                                                                |
|--------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>success_url</code> | <code>https://app.com/success?</code><br><code>session_id={CHECKOUT_SESSION_ID}</code> | <code>{CHECKOUT_SESSION_ID}</code><br>Stripe নিজে replace করে দেয় |
| <code>cancel_url</code>  | <code>https://app.com/cancel</code>                                                    | User checkout থেকে "Back"<br>চাপলে এখানে যায়                      |

## 3.2 🛎 Webhook Handle করা

ধাপ ১ — SIGNATURE VERIFY করা

```

import stripe
from django.http import HttpResponse
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def stripe_webhook(request):

```

```

payload      = request.body
sig_header   = request.META.get('HTTP_STRIPE_SIGNATURE')
webhook_secret = stripe_cfg.get('STRIPE_WEBHOOK_SECRET')

try:
    # signature verify করে – fake request ঠেকায়
    event = stripe.Webhook.construct_event(
        payload, sig_header, webhook_secret
    )
except ValueError:
    return HttpResponse(status=400) # invalid payload
except stripe.error.SignatureVerificationError:
    return HttpResponse(status=400) # signature মিলছে না – fake!

```

## ধাপ ২ — CHECKOUT.SESSION.COMPLETED EVENT HANDLE করা

```

event_type = event['type']
event_data = event['data']['object']

if event_type == 'checkout.session.completed':
    metadata = event_data.get('metadata') or {}
    user_id   = metadata.get('user_id')
    mc_id     = metadata.get('micro_credential_id')
    amount    = event_data.get('amount_total', 0) / 100
    session_id = event_data.get('id')

    # duplicate processing ঠেকাও (webhook একাধিকবার আসতে পারে)
    if
PaymentTransaction.objects.filter(stripe_payment_intent_id=session_id).exists():
    return HttpResponse(status=200)

    user = User.objects.get(id=user_id)
    payment = PaymentTransaction.objects.create(
        user=user, amount=amount, status='completed',
        stripe_payment_intent_id=session_id,
    )
    MCAccess.objects.create(user=user, micro_credential_id=mc_id,
                            is_active=True, payment_transaction=payment)

    # email পাঠাও ...

```

## ধাপ ৩ — PAYMENT\_INTENT.PAYMENT\_FAILED EVENT HANDLE করা

```

elif event_type == 'payment_intent.payment_failed':
    intent_id = event_data.get('id')
    error_msg = (event_data.get('last_payment_error') or {}).get('message')

    PaymentTransaction.objects.filter(
        stripe_payment_intent_id=intent_id
    )

```

```
).update(status='failed')

# চাইলে user কে "payment failed" ইমেইল পাঠাতে পারো
# logger.warning(f"Payment failed: {error_msg}")
```

## ধাপ ৪ — 3DS এর জন্য PAYMENT\_INTENT.REQUIRES\_ACTION HANDLE করা

### নোট:

Stripe

### Checkout Session

ব্যবহার করলে এই event সাধারণত তোমার backend এ handle করা লাগে না — কারণ Stripe এর hosted page নিজেই 3DS challenge সামলে নেয়। কিন্তু raw PaymentIntent API (V1 approach, embedded card form) ব্যবহার করলে এই event টা track করা useful, যাতে monitor করতে পারো কতজন user 3DS challenge face করছে।

```
elif event_type == 'payment_intent.requires_action':
    intent_id = event_data.get('id')
    # শুধু logging/analytics এর জন্য track করা – action নেওয়ার দরকার নেই
    # কারণ Stripe নিজেই client কে OTP page দেখাবে
    PaymentTransaction.objects.filter(
        stripe_payment_intent_id=intent_id
    ).update(status='pending_3ds')

    return HttpResponse(status=200) # Stripe কে বলো "পেয়েছি, ঠিক আছে"
```

## 3.3 Environment Setup

```
# .env ফাইলে (Backend)
STRIPE_SECRET_KEY=sk_test_51Abc...xyz
# শুধু Backend এ থাকবে, কখনো Frontend এ না
STRIPE_WEBHOOK_SECRET=whsec_a1B2c3D4... # webhook signature verify করতে
লাগে
STRIPE_PUBLISHABLE_KEY=pk_test_51Abc...xyz # Frontend এ পাঠানো যায়, safe to
expose
```

| Key                    | কোথায় থাকবে       | কাজ                                                      |
|------------------------|--------------------|----------------------------------------------------------|
| STRIPE_SECRET_KEY      | শুধু Backend       | Stripe API কল করতে (Session তৈরি, PaymentIntent ইত্যাদি) |
| STRIPE_WEBHOOK_SECRET  | শুধু Backend       | Webhook signature verify করতে (fake request ঠেকাতে)      |
| STRIPE_PUBLISHABLE_KEY | Backend + Frontend | Frontend এ Stripe.js initialize করতে                     |

## 8. 🖥️ Frontend Implementation Guide (React/JavaScript)

### 4.1 ⚙️ Stripe.js এবং Stripe Elements Setup

Checkout Session approach এ পুরো card form বানাতে হয় না, কিন্তু [@stripe/stripe-js](#) লোড করা ভালো অভ্যাস (redirect হেল্পার ফাংশনের জন্য)।

```
npm install @stripe/stripe-js
```

```
// lib/stripe.js
import { loadStripe } from '@stripe/stripe-js';

export const stripePromise =
  loadStripe(process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY);
```

### 4.2 ☎️ Backend API Call করে session\_url নেওয়া

```
async function startCheckout(microCredentialId) {
  const token = localStorage.getItem('access_token');

  const res = await fetch('/api/v2/enrollment/', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ micro_credential_id: microCredentialId }),
  });

  const data = await res.json();

  if (data.success) {
    return data.checkout.checkout_url; // এই URL এ redirect করবো
  }
  throw new Error(data.message);
}
```

## 4.3 Redirect করে Stripe Page এ পাঠানো

দুইটা উপায় আছে:

1. সরাসরি redirect (সবচেয়ে সহজ, recommended):

```
window.location.href = checkout_url
```

2. stripe.redirectToCheckout() (legacy helper, session id দিয়ে):

নিচে দুইটাই দেখানো হলো

```
// Option A – সরাসরি redirect (সহজ ও recommended)
function handleBuyClick(microCredentialId) {
  startCheckout(microCredentialId)
    .then((url) => { window.location.href = url; })
    .catch((err) => setError(err.message));
}

// Option B – stripe.redirectToCheckout() দিয়ে (session_id লাগবে)
import { stripePromise } from '../lib/stripe';

async function handleBuyClickV2(microCredentialId) {
  const token = localStorage.getItem('access_token');
  const res = await fetch('/api/v2/enrollment/', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${token}` },
    body: JSON.stringify({ micro_credential_id: microCredentialId }),
  });
  const data = await res.json();

  const stripe = await stripePromise;
  const { error } = await stripe.redirectToCheckout({
    sessionId: data.checkout.session_id,
  });
  if (error) setError(error.message);
}
```

## 4.4 🛡️ 3DS OTP Confirmation — Stripe নিজেই Handle করে কীভাবে

### ✅ ভালো খবর:

Checkout Session ব্যবহার করলে তোমাকে `handleCardAction()` , `confirmCardPayment()` বা কোনো popup manually খোলা লাগে না। User checkout.stripe.com পেজেই থাকে, আর 3DS দরকার হলে Stripe সেই একই পেজে bank এর OTP challenge iframe হিসেবে দেখিয়ে দেয়। User OTP দিলে Stripe নিজেই confirm করে এবং success/cancel URL এ redirect করে দেয়। Frontend কোডে অতিরিক্ত কিছু লেখা লাগে না!

## 4.5 ✅ Success Page এ session\_id দিয়ে Payment Verify করা

```
// pages/enrollment/success.jsx
import { useEffect, useState } from 'react';
import { useSearchParams } from 'next/navigation';

export default function SuccessPage() {
  const params = useSearchParams();
  const sessionId = params.get('session_id');
  const [status, setStatus] = useState('checking');

  useEffect(() => {
    if (!sessionId) return;

    // Backend এ একটা verify endpoint কল করো (session_id দিয়ে)
    fetch(`/api/v2/enrollment/verify/?session_id=${sessionId}`, {
      headers: { Authorization: `Bearer ${localStorage.getItem('access_token')}` }
    })
      .then((res) => res.json())
      .then((data) => setStatus(data.success ? 'success' : 'pending'))
      .catch(() => setStatus('pending'));
  }, [sessionId]);

  if (status === 'checking') return <p>যাচাই করা হচ্ছে...</p>;
  if (status === 'success')
    return (
      <div>
        <h1>🎉 Payment Successful!</h1>
        <p>আপনার Micro-Credential কেনা হয়েছে। এখনই শেখা শুরু করুন!</p>
      </div>
    );
  return (
    <div>
      <h1>⌚ Processing...</h1>
      <p>আপনার পেমেন্ট প্রসেস হচ্ছে। কয়েক সেকেন্ড অপেক্ষা করুন — webhook থেকে access grant হবে!</p>
    </div>
  );
}
```



```

p>
  </div>
);
}

```

#### 4.6 ⚠ Error Handling — card declined, 3DS failed, expired card

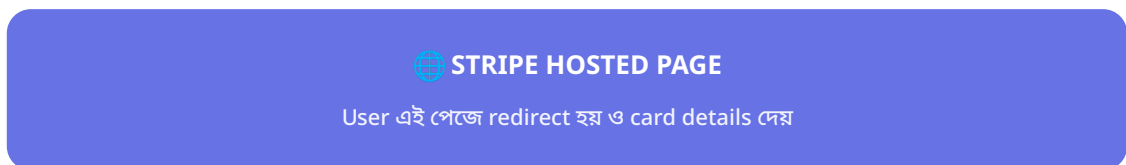
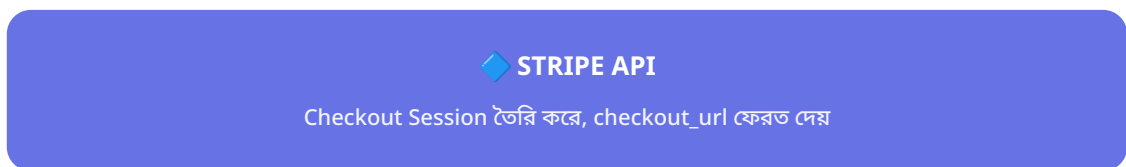
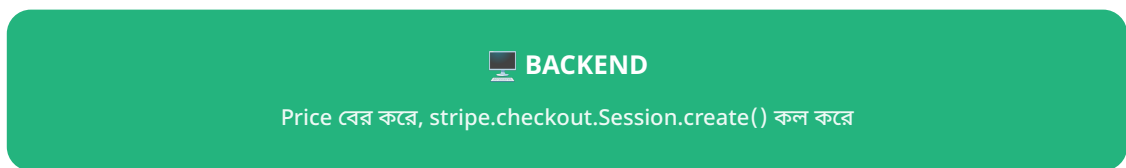
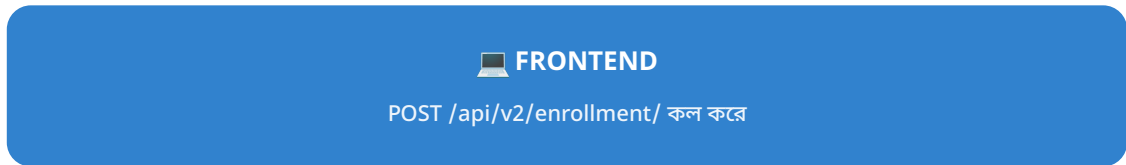
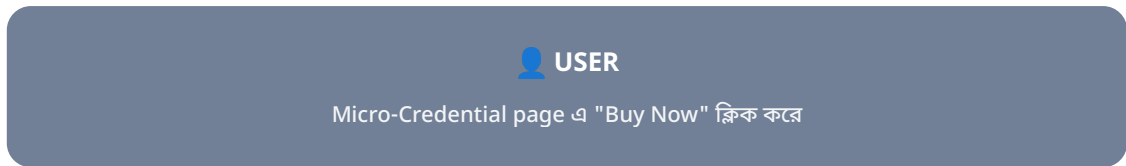
```

// cancel.jsx বা error boundary তে
function getErrorMessage(errorCode) {
  const messages = {
    card_declined: 'আপনার card declined হয়েছে। অন্য card দিয়ে চেষ্টা করুন।',
    insufficient_funds: 'আপনার account এ পর্যাপ্ত ব্যালেন্স নেই।',
    expired_card: 'আপনার card এর মেয়াদ শেষ হয়ে গেছে।',
    incorrect_cvc: 'CVC নম্বর ভুল দেওয়া হয়েছে।',
    authentication_required: '3D Secure authentication ব্যর্থ হয়েছে। আবার চেষ্টা করুন।',
    processing_error: 'একটা সাময়িক সমস্যা হয়েছে। আবার চেষ্টা করুন।',
  };
  return messages[errorCode] || 'পেমেন্ট ব্যর্থ হয়েছে। আবার চেষ্টা করুন।';
}

// cancel_url এ redirect হলে (user "Back" চাপলে বা payment ব্যর্থ হলে)
export default function CancelPage() {
  return (
    <div>
      <h1>❌ Payment Cancelled</h1>
      <p>পেমেন্ট সম্পন্ন হয়নি। আপনি আবার চেষ্টা করতে পারেন।</p>
      <button onClick={() => router.back()}>আবার চেষ্টা করুন</button>
    </div>
  );
}

```

## ৫. 🌐 সম্পূর্ণ End-to-End Flow Diagram



 **3DS Required?**



✓ **PAYMENT SUCCESS**

Stripe payment charge করে



📡 **STRIPE WEBHOOK → BACKEND**

checkout.session.completed event পাঠায়



📄 **ENROLLMENT/ACCESS তৈরি**

Backend signature verify করে, DB update করে, email পাঠায়



🎉 **USER → SUCCESS PAGE**

session\_id দিয়ে redirect, frontend verify করে

## ৬. Test Cards এবং Scenarios

Stripe Test mode এ নিচের card গুলো ব্যবহার করে বিভিন্ন scenario টেস্ট করা যায়। সব ক্ষেত্রে **যেকোনো future expiry** (যেমন 12/34), **যেকোনো 3-digit CVC** (যেমন 123), এবং **যেকোনো postal code** ব্যবহার করা যাবে।

| Card Number         | Scenario                 | Expected Result                                                                                            |
|---------------------|--------------------------|------------------------------------------------------------------------------------------------------------|
| 4242 4242 4242 4242 | Normal payment           |  Success                |
| 4000 0025 0000 3155 | 3DS Required             |  OTP page আসবে          |
| 4000 0000 0000 9995 | Insufficient funds       |  Declined               |
| 4000 0000 0000 0069 | Expired card             |  Declined               |
| 4000 0000 0000 0127 | Incorrect CVC            |  Declined               |
| 4000 0082 6000 3178 | 3DS Supported (optional) |  আসতেও পারে, নাও পারে |
| 4000 0027 6000 3184 | 3DS Required, then fails |  Auth ব্যর্থ হবে      |

### কীভাবে টেস্ট করবে

Stripe Dashboard কে *Test mode* এ রেখে `STRIPE_SECRET_KEY=sk_test_...` ব্যবহার করো। Test card গুলো real bank এ চার্জ করবে না — শুধু Stripe এর sandbox এ simulate হবে।

## ৭. 🛠️ Common Errors এবং Solutions

### ❌ Error: "No signatures found matching the expected signature for payload"

কারণ:

`STRIPE_WEBHOOK_SECRET` ভুল, অথবা Stripe CLI ও production webhook এর secret আলাদা।

সমাধান:

Stripe Dashboard থেকে সঠিক webhook secret কপি করো, অথবা লোকাল টেস্টের জন্য `stripe listen` কমান্ড থেকে দেওয়া secret ব্যবহার করো।

### ❌ Error: Webhook Localhost এ কাজ করছে না

কারণ:

Stripe এর সার্ভার তোমার localhost এ সরাসরি request পাঠাতে পারে না (internet থেকে access করা যায় না)।

সমাধান:

`stripe listen --forward-to localhost:8000/enrollment/stripe/webhook/` কমান্ড দিয়ে Stripe CLI ব্যবহার করো — এটা webhook গুলো তোমার লোকাল সার্ভারে forward করে দেবে।

### ❌ Error: Payment succeeded কিন্তু user access পায়নি

কারণ:

Webhook event backend এ পৌঁছায়নি, অথবা metadata তে `user_id` / `micro_credential_id` ঠিকমতো পাঠানো হয়নি।

সমাধান:

Stripe Dashboard → Developers → Webhooks এ গিয়ে event log চেক করো। Checkout Session তৈরি করার সময় metadata ঠিকমতো সেট করা হয়েছে কিনা যাচাই করো।

### ❌ Error: Duplicate Enrollment তৈরি হচ্ছে

#### কারণ:

Stripe একই webhook event একাধিকবার পাঠাতে পারে (at-least-once delivery guarantee)।

#### সমাধান:

Webhook handler এ প্রসেস করার আগে চেক করো `stripe_payment_intent_id` / `session_id` আগে থেকেই DB তে আছে কিনা — থাকলে skip করো (idempotency)।

### ❌ Error: "This PaymentIntent's client\_secret does not match..."

#### কারণ:

Test mode ও Live mode এর key mix হয়ে গেছে (যেমন `sk_test_` দিয়ে `pk_live_` ব্যবহার করা)।

#### সমাধান:

নিশ্চিত করো Secret key, Publishable key, এবং Webhook secret — তিনটাই একই mode (test বা live) থেকে এসেছে।

### ❌ Error: Card declined generic error দেখাচ্ছে, কারণ বোঝা যাচ্ছে না

#### কারণ:

Stripe security কারণে decline এর সঠিক কারণ সবসময় merchant কে জানায় না।

#### সমাধান:

`payment_intent.last_payment_error.decline_code` চেক করো (যেমন `insufficient_funds`, `expired_card`) এবং সেই অনুযায়ী user friendly বাংলা মেসেজ দেখাও।

### ❌ Error: CORS Error Frontend থেকে API কল করলে

#### কারণ:

Django backend এ Frontend এর domain `CORS_ALLOWED_ORIGINS` এ যুক্ত করা নেই।

#### সমাধান:

`settings.py` এ `django-cors-headers` ব্যবহার করে Frontend URL whitelist করো।

**✖ Error: Success page এ redirect হলো কিন্তু "pending" দেখাচ্ছে**

**কারণ:**

Webhook এখনো process হয়নি — redirect ও webhook সমান্তরাল ঘটনা, redirect আগে হতেই পারে।

**সমাধান:**

Success page এ কয়েক সেকেন্ড পরপর polling করো (backend এ `session_id` দিয়ে status চেক),  
অথবা loading state দেখাও যতক্ষণ না webhook প্রসেস হয়।



## ৮. 📋 Quick Reference Card

### 🔑 Environment Variables

```
STRIPE_SECRET_KEY=sk_test_...  
STRIPE_WEBHOOK_SECRET=whsec_...  
STRIPE_PUBLISHABLE_KEY=pk_test_...
```

### 🌐 API Endpoints

| Method | URL                         |
|--------|-----------------------------|
| POST   | /api/v2/enrollment/         |
| POST   | /enrollment/stripe/webhook/ |
| GET    | /enrollment/plans/          |

### 📡 Webhook Events

| Event                          | মানে                             |
|--------------------------------|----------------------------------|
| checkout.session.completed     | পেমেন্ট<br>সফল,<br>access<br>দাও |
| payment_intent.payment_failed  | পেমেন্ট<br>ব্যর্থ                |
| payment_intent.requires_action | 3DS<br>pending                   |

### 🔪 Test Cards

| Card                   | ফলাফল                   |
|------------------------|-------------------------|
| 4242 4242 4242<br>4242 | ✅ Success               |
| 4000 0025 0000<br>3155 | 🔒 3DS                   |
| 4000 0000 0000<br>9995 | ❌ Insufficient<br>funds |
| 4000 0000 0000<br>0069 | ❌ Expired               |

### ⚡ মনে রাখার মূল বিষয়

- ✅ Card details কখনো backend এ যায় না — Stripe hosted page নিজেই নেয়
- ✅ 3DS Checkout Session এ automatic — কোনো popup কোড লাগে না
- ✅ metadata দিয়েই webhook এ user/item চেনা যায়

- ✓ Webhook signature সবসময় verify করো — fake request ঠেকাতে
- ✓ Duplicate webhook ঠেকাতে session\_id দিয়ে idempotency চেক করো
- ✓ Real access grant হয় webhook থেকে, redirect থেকে না
- ✓ Local testing এ `stripe listen --forward-to` ব্যবহার করো

---

🎓 এই গাইডটি LifeChoice Backend প্রজেক্টের Stripe Checkout ও 3D Secure ইন্টিগ্রেশন শেখার জন্য বাংলায় তৈরি করা হয়েছে।